AgentForge

Clinical Co-Pilot - Week 2

Multimodal Evidence Agent: seeing clinical documents, routing work, and gating changes with evals

Project Requirements Document

Gauntlet AI - Austin Admission Track

How to Use This Assignment

Your Week 1 agent already reads structured OpenEMR data, attributes claims, logs tool behavior, and has a starter eval suite. This week, you add two new capabilities: the agent can read real-world clinical documents, and it can route work across a small multi-agent graph without losing grounding.

GATE	Eval-driven CI is non-negotiable. A working demo that cannot block regressions has not met the Week 2 standard.

The Scenario

A primary care physician is prepping for a follow-up visit. The chart has structured OpenEMR data, but the important recent information is buried in a scanned lab PDF and a patient intake form uploaded by the front desk. The physician asks: What changed, what should I pay attention to, and what evidence supports the recommendation?

Your Week 2 Clinical Co-Pilot must ingest the lab PDF and intake form, extract structured facts with citations, retrieve relevant guideline evidence, and return a grounded answer. The answer should be useful even if the document scan is imperfect, the patient record is incomplete, or the user asks a follow-up question.

WHY THIS MATTERS	Clinical agents fail when they cannot handle the messy inputs clinicians actually receive. Week 2 is about making the agent see, keeping the architecture small enough to reason about, and proving quality through automated evals.

The Hard Problems

Vision extraction without invention

A VLM can read a scanned form, but it can also hallucinate field labels or overstate confidence. Your schema, source links, and verification strategy must make unsupported extracted facts visible.

Evidence grounding

Every answer must separate patient-record facts from guideline evidence. A medication or lab claim is not acceptable unless it points back to a source.

Multi-agent architecture

The goal is to give multiple workers clear responsibilities and make the supervisor's routing decisions inspectable.

Eval-driven development

You will use a 50-case golden set with boolean rubrics. The CI gate must catch regressions before they reach the demo.

FHIR and OpenEMR integrity

Uploaded documents and derived observations must round-trip through OpenEMR without creating duplicate or untraceable records.

HIPAA-minded development

Use only demo or synthetic data. Do not log raw PHI. Treat prompts, extracted fields, document images, traces, and screenshots as sensitive.

The Codebase

Build on the Week 1 fork, auth flow, tool layer, verification strategy, observability, and eval harness. Good Week 1 architecture should compound here; technical debt from Week 1 should be documented and resolved before adding new surface area.

The Week 2 repo may stay inside the same OpenEMR fork, but your README must clearly separate Week 1 baseline behavior from Week 2 multimodal behavior. Graders should be able to run the core Week 2 flow without guessing which branch, environment variable, or service is required.

Project Schedule

Hard gates. This is a one-week sprint with four checkpoints. All times are Central (Austin).

Checkpoint	Deadline	Focus
Architecture Defense	4 hours	Document schemas, RAG and eval design, security concerns
MVP	Tuesday @ 11:59PM	Lab PDF and intake form ingestion working locally; first extraction and first evidence retrieval demo.
Early Submission	Thursday @ 11:59PM	Supervisor plus 2 workers, 50-case eval suite, PR-blocking CI, deployed app, demo video.
Final	Sunday @ Noon	Production-ready Week 2 agent, source-grounded demo, cost/latency report, and interview readiness.

MVP: Recommended Steps

The MVP is not a full medical-document AI platform. It is a controlled expansion of the Week 1 agent into two document types, two workers, and one regression gate.

Name	Deliverable
Ingest two document types	Upload and extract a lab PDF and an intake form using strict schemas.
Build basic hybrid RAG	Small guideline corpus indexed with keyword+dense retrieval and Cohere rerank or equivalent.
Add supervisor + 2 workers	Supervisor routes to intake-extractor and evidence-retriever with logged handoffs.
Gate with eval-driven CI	50-case golden set, boolean rubrics, PR-blocking Git Hook.
Integrate and demo	Deployed app, source-grounded UI, latency/cost report, walkthrough video.

Stage 1 - Ingest Lab PDF and Intake Form

Implement a document ingestion flow that accepts a file, associates it with a patient, stores the source document in OpenEMR, extracts structured JSON, and links every derived fact back to the source. Required document types are a lab PDF and an intake form.

Stage 2 - Build Basic Hybrid RAG

Create a small clinical-guideline corpus relevant to your user profile. Use keyword plus vector retrieval, rerank the candidate chunks, and return evidence snippets with source metadata. ColQwen2 and multi-vector indexing are stretch; the core requirement is a reliable hybrid retriever.

Stage 3 - Add Supervisor + 2 Workers

Implement a small graph: one supervisor, one intake-extractor worker, and one evidence-retriever worker. The supervisor should decide when extraction is needed, when evidence retrieval is needed, and when the final answer is ready. Keep handoffs explicit.

Stage 4 - Build the Eval Gate

Create 50 synthetic or demo cases that exercise extraction, evidence retrieval, citations, refusals, and missing-data behavior. Use boolean rubrics, not 1-10 ratings. CI must fail on meaningful regression.

Stage 5 - Integrate, Deploy, and Defend

Expose the Week 2 flow in the deployed app, capture observability traces, record a demo, and prepare to explain why each capability maps back to the Week 1 user and workflow.

Core Agent Requirements

1. Document ingestion and extraction

Implement attach_and_extract(patient_id, file_path, doc_type) or an equivalent tool. It must support lab_pdf and intake_form. It must store the source document in OpenEMR, return strict-schema JSON, and persist derived facts as appropriate FHIR resources or OpenEMR records.

2. Structured schemas

Use Pydantic, Zod, or equivalent strict schemas. Required lab fields include at least test name, value, unit, reference range, collection date, abnormal flag, and source citation. Required intake fields include demographics fields, chief concern, current medications, allergies, family history, and source citation.

3. Basic hybrid RAG plus rerank

Index a small clinical-guideline corpus. Retrieve with sparse+dense search, rerank candidate chunks with Cohere Rerank or an equivalent reranker, and feed only the top grounded evidence to the answer model.

4. Supervisor plus two workers

Use LangGraph, the OpenAI Agents SDK, or another inspectable orchestration framework. Required workers are intake-extractor and evidence-retriever. A critic agent is extension work, not core.

5. Citation contract

Every clinical claim in the final response must include machine-readable citation metadata. Minimum citation shape: {source_type, source_id, page_or_section, field_or_chunk_id, quote_or_value}. A visual PDF bounding-box overlay is required.

6. Eval-driven CI gate

Build a 50-case golden set and a PR-blocking Git Hook. Boolean rubric categories must include schema_valid, citation_present, factually_consistent, safe_refusal, and no_phi_in_logs. The build must fail if any category regresses by more than 5% or drops below the pass threshold.

7. Observability and cost tracking

Each encounter must log tool sequence, latency by step, token usage, cost estimate, retrieval hits, extraction confidence, and eval outcome. Logs must not contain raw PHI.

HARD GATE	During grading, we will introduce a small regression and confirm your CI gate fails. If the eval gate does not block the regression, the Week 2 build does not pass.

Core Deliverables

- Two document types: lab PDF and intake form.
- One supervisor and two workers: intake-extractor and evidence-retriever.
- Basic hybrid RAG plus rerank over a small guideline corpus.
- 50-case golden dataset with boolean rubrics.
- PR-blocking eval CI and an observable deployed demo.
- Critic agent that rejects uncited claims or unsafe action suggestions.
- Click-to-source UI for citation snippets, with a simple document preview.
- A third document type such as referral fax or medication list.
- Lab trend chart widget that uses extracted Observation data.
- Contextual retrieval improvements such as better chunking, query rewriting, or domain-specific filters.

Engineering Requirements

The following requirements apply in addition to the project-specific deliverables above. They are graded alongside the core submission and are not optional.

API and event contracts, schema evolution, migration safety, data authority. Every interface between Week 2 components — document ingestion, RAG retrieval, supervisor handoffs, FHIR writes — must have a typed contract (Pydantic/Zod schema or equivalent). Any schema change from Week 1 must be accompanied by a migration note. Data authority must be explicit: one source of truth per data type, no silent overwrites.

Logs, metrics, traces, dashboards — SLOs, queues, retries, timeouts, circuit breakers. Extend Week 1 observability to cover Week 2 flows: document ingestion latency, extraction confidence per document, RAG retrieval hit rate, supervisor routing decisions, and per-worker latency. Add SLOs for document ingestion (p95 < X seconds) and evidence retrieval. All outbound LLM and retrieval calls must have timeouts and retry logic.

Produce canonical API/event/schema contracts from cleaned requirements. The extraction schemas (lab_pdf, intake_form) are the canonical contracts. Do not let raw VLM output bypass schema validation. The schema is the source of truth — not what the model happens to return.

Every request or event carries a correlation ID across service boundaries. The correlation ID from Week 1 must propagate into Week 2 document ingestion flows, worker handoffs, and FHIR writes. A full multi-agent trace must be reconstructable from the correlation ID alone.

Structured logs searchable by case ID, event ID, and correlation ID. Extend Week 1 log schema to cover Week 2 events: document ingestion start/complete, extraction outcome per field, retrieval hit/miss, worker handoff, and eval run outcome. All logs must remain PHI-free.

Dashboards: request count, error count, latency, queue depth, event retries, decision outcomes. Add Week 2 metrics to the Week 1 dashboard: document ingestion count, extraction field-level pass rate, retrieval hit rate, worker routing decisions, eval pass/fail rate per category. The dashboard should tell a grader whether the system is healthy without reading logs.

CI pipeline: build, lint/typecheck, tests, coverage, dependency audit, security scan. The Week 1 eval CI gate must be extended to cover Week 2: add schema validation tests, contract tests for the supervisor-worker interface, and extraction regression tests to the PR-blocking suite. Dependency audit and security scan must run on every PR.

Testing strategy and implementation. Document your testing strategy in W2_ARCHITECTURE.md: what is unit-tested (schema validators, tool functions), what is integration-tested (ingestion flow, RAG pipeline), what is evaluated via the golden set (agent behavior), and what is not tested and why. Every test must have a documented failure mode it guards against.

Observability, debugging, and incident response. Extend your ARCHITECTURE.md to cover Week 2 failure modes: document ingestion failures, extraction schema violations, RAG retrieval returning no results, and supervisor routing errors. Each entry must describe how to identify the failure in logs and what the recovery action is.

Commonly used API calls in Postman, Bruno, or equivalent runnable API collection. Update the Week 1 API collection to include Week 2 endpoints: document upload, extraction status, evidence retrieval, and the full Week 2 agent flow. Graders must be able to run any Week 2 workflow from the collection.

Baseline CPU, memory, latency, and throughput profiles. Record baseline metrics for Week 2 flows: document ingestion, extraction, RAG retrieval, and full multi-agent run. Compare against Week 1 baselines to verify the new components have not introduced unexpected regressions in shared paths.

Consistent structured logging. Week 2 logging must follow the same structured format established in Week 1. No plain-text log output from Week 2 components. Extend the log schema for new event types; do not create a parallel logging convention.

Correlation/request IDs across services. Propagate the correlation ID into all Week 2 worker invocations, VLM calls, retrieval calls, and FHIR writes. A grader must be able to reconstruct a full Week 2 request trace using only the correlation ID.

Distributed tracing for internal diagnosis. Extend Week 1 distributed tracing to cover the Week 2 supervisor/worker graph. Each worker invocation must be a child span of the supervisor span. Extraction and retrieval sub-calls must be traceable within their worker spans.

Separate /health and /ready endpoints; readiness must validate meaningful dependencies. Update /ready to check Week 2 dependencies: document storage, vector index, and reranker API reachability. /ready should return degraded status if any dependency is unavailable, not just a binary up/down.

Dashboard and alert definitions. Add Week 2 alerts: extraction failure rate, RAG retrieval latency, eval regression detection (more than 5% drop in any category triggers an alert). Alerts must be documented with expected response actions.

OpenAPI 3.0 / Swagger definitions. Publish an OpenAPI 3.0 spec for all Week 2 HTTP endpoints. The spec must be committed to the repo and kept in sync with the implementation. Contract tests must verify the implementation matches the spec.

Integration tests with fixtures and stubs. Write integration tests that exercise the full ingestion-to-answer path using fixture documents (stored PDFs and form images) and stubbed LLM/VLM responses. These tests must pass in CI without live API access.

Data modeling, ingestion, validation, lineage, access control, reporting, data quality. Document the data model for Week 2 artifacts: extracted lab observations, intake facts, guideline chunks, and citation records. Each type must have a defined owner (which system is authoritative), lineage (where it came from), access control (who can read/write it), and validation rules.

Privacy and records implications of analytics workflows. Audit the Week 2 observability data for PHI leakage. Traces, logs, eval datasets, and cost reports must not contain patient identifiers, raw document text, or extracted clinical values. Document the scrubbing approach and verify it in CI with a PHI-detection check.

Backup and recovery plan — automatic and manual. Document how extracted documents, derived FHIR records, and the eval golden set are backed up. Describe the manual recovery procedure if automated backup fails. Include RPO and RTO estimates. The eval golden set in particular must be reproducible from the repo alone — it should not live only in a database that has no recovery path.

Deliverable	Requirements
GitLab Repository	Week 1 fork with Week 2 changes, setup guide, deployed link, and clear environment-variable documentation.
W2 Architecture Doc	A ./W2_ARCHITECTURE.md file explaining the document ingestion flow, worker graph, RAG design, eval gate, risks, and tradeoffs.
Schemas	Pydantic/Zod schemas for lab_pdf and intake_form, including source citation fields and validation tests.
Eval Dataset	50 synthetic/demo cases with expected behavior, boolean rubrics, judge configuration, and results.
CI Evidence	Git Hook or equivalent that runs the eval suite and blocks regressions.
Demo Video	3-5 minutes showing document upload, extraction, evidence retrieval, citations, eval results, and observability.
Cost and Latency Report	Actual dev spend, projected production cost, p50/p95 latency, and bottleneck analysis.
Deployed Application	Publicly accessible deployed app with the Week 2 core flow working.

Common Pitfalls and Watch-Outs

- Trying to support five document types before two work reliably.
- Using a VLM answer directly without schema validation or source metadata.
- Letting the supervisor become a black box. Handoffs must be logged and explainable.
- Using llm-as-a-judge without clear rubric. Use boolean rubrics so failures are actionable.
- Logging raw document text, patient identifiers, or screenshots to SaaS observability tools.

Final Note

Week 2 is not a contest to integrate the most AI frameworks. It is a test of whether you can add multimodal inputs, keep the agent architecture comprehensible, and prove quality with a CI gate. The best submissions will feel narrower than the original spec and stronger because of it.